

中山大學
SUN YAT-SEN UNIVERSITY

并行程序设计与算法实验

Lab5-基于 OpenMP 的并行矩阵乘法

姓 名 林宏宇

学 号 23320093

学 院 计算机学院

专 业 计算机科学与技术

2026 年 5 月 6 日

1 实验目的

- 掌握 OpenMP 编程的基本流程
- 掌握常见的 OpenMP 编译指令和运行库函数
- 分析调度方式对多线程程序的影响

2 实验内容

- 使用 OpenMP 实现并行通用矩阵乘法
- 设置不同线程数量（1-16）、矩阵规模（128-2048）、调度方式
  - 调度方式包括默认调度、静态调度、动态调度
- 根据运行时间，分析程序并行性能
- 选做：根据运行时间，对比使用 OpenMP 实现并行矩阵乘法与使用 Pthreads 实现并行矩阵乘法的性能差异，并讨论分析。

3 实验结果

表 1: 默认调度

线程数	矩阵规模				
	128	256	512	1024	2048
1	4.173ms	17.872ms	79.756ms	621.700ms	7005.643ms
2	2.271ms	15.983ms	57.082ms	322.121ms	3729.484ms
4	1.433ms	8.319ms	37.781ms	171.441ms	1993.081ms
8	1.253ms	4.607ms	29.509ms	94.010ms	1036.010ms
16	1.828ms	3.304ms	16.776ms	58.566ms	548.514ms

表 2: 静态调度

线程数	矩阵规模				
	128	256	512	1024	2048
1	4.165ms	19.329ms	79.715ms	614.283ms	7192.747ms
2	2.215ms	16.124ms	56.845ms	323.962ms	4205.540ms
4	1.569ms	10.893ms	35.549ms	171.367ms	1878.935ms
8	1.153ms	4.664ms	26.187ms	93.836ms	952.940ms
16	1.554ms	3.434ms	16.534ms	59.603ms	487.190ms

表 3: 动态调度

线程数	矩阵规模				
	128	256	512	1024	2048
1	4.585ms	22.008ms	87.978ms	688.606ms	7356.722ms
2	3.318ms	17.350ms	60.035ms	389.261ms	3973.898ms
4	2.909ms	10.383ms	40.709ms	206.601ms	2216.356ms
8	2.022ms	7.160ms	28.944ms	128.815ms	1111.109ms
16	2.029ms	4.450ms	17.734ms	74.206ms	583.112ms

4 实验分析

4.1 并行性能分析

由表 1–表 3 的实验数据可以观察到以下并行性能特征：

加速比分析：以默认调度 (2048 矩阵) 为例，单线程运行时间为 7005.643ms，2 线程为 3729.484ms（加速比 1.88x），4 线程为 1993.081ms（加速比 3.52x），8 线程为 1036.010ms（加速比 6.76x），16 线程为 548.514ms（加速比 12.77x）。随着线程数增加，程序运行时间显著减少，展现出良好的可扩展性。但由于线程创建和同步开销，以及缓存/内存带宽争用等因素，加速比并非线性增长，尤其在较小矩阵规模下更为明显——例如 128 矩阵从 1 线程到 16 线程的加速比仅为 2.28x，远低于理论值。

矩阵规模的影响：矩阵规模越大，并行加速效果越明显。以默认调度 16 线程为例，128 矩阵加速比为 2.28x，256 矩阵加速比为 5.41x，512 矩阵加速比为 4.75x，1024 矩阵加速比为 10.62x，2048 矩阵加速比为 12.77x。总体上，矩阵规模增大使每个线程的计算负载更饱满，有效摊薄了并行开销；在本实验范围内未出现明显的带宽饱和现象。

4.2 调度方式对比分析

对比三种调度方式的表现：

默认调度与静态调度：两种调度方式的性能非常接近。OpenMP 标准并未强制规定默认调度的具体策略，具体实现可能不同；在 GCC 等常见实现中，规则的 `for` 循环默认采用 `static` 调度，因此与显式 `static` 在性能上表现接近。本实验中默认调度与静态调度确实接近，这与上述实现习惯一致，也反映出迭代负载较均衡、静态划分开销最低。

动态调度：动态调度在单线程下运行时间略长（2048 矩阵：7356.722ms vs 7005.643ms），因为动态调度引入了额外的任务分配开销。但在多线程场景下（如 16 线程 2048 矩阵：583.112ms vs 548.514ms），动态调度仍略慢，主要由于调度开销增加；矩阵乘法各迭代负载均匀，因此动态调度的负载均衡优势在本实验中未能体现。在负载不均衡的实际应用场景中，动态调度才能发挥其优势。

小结：对于负载均衡的矩阵乘法任务，推荐使用默认/静态调度以获得最优性能；当各迭代计算量差异较大时，动态调度可通过按需分配任务来避免负载不均。

4.3 选做：OpenMP 与 Pthreads 性能对比

为对比 OpenMP 与 Pthreads 的性能差异，使用相同的矩阵乘法算法（三层循环，ikj 顺序）分别用 Pthreads 实现并进行相同规模的测试。Pthreads 版本手动将矩阵 A 的行均匀分配给各线程，每个线程独立计算所分配行的 C 矩阵结果。

4.3.1 对比数据

表 4 展示了 Pthreads 实现在各线程数和矩阵规模下的运行时间，表 5 对比了 OpenMP（默认调度）与 Pthreads 在 2048 规模下的运行时间与加速比。

表 4: Pthreads 并行矩阵乘法运行时间

线程数	矩阵规模				
	128	256	512	1024	2048
1	4.747ms	24.722ms	101.874ms	701.367ms	8314.071ms
2	2.616ms	16.208ms	59.925ms	366.635ms	4269.847ms
4	1.574ms	11.260ms	40.019ms	197.649ms	2175.121ms
8	1.296ms	4.964ms	27.837ms	105.374ms	1166.436ms
16	1.824ms	5.251ms	17.381ms	66.235ms	617.557ms

表 5: OpenMP(默认调度) 与 Pthreads 对比 (2048 规模)

线程数	运行时间 (ms)		加速比	
	OpenMP	Pthreads	OpenMP	Pthreads
1	7005.643	8314.071	1.00x	1.00x
2	3729.484	4269.847	1.88x	1.95x
4	1993.081	2175.121	3.52x	3.82x
8	1036.010	1166.436	6.76x	7.13x
16	548.514	617.557	12.77x	13.46x

4.3.2 性能分析

从表 5 数据可以得出以下结论：

绝对性能：在单线程场景下，OpenMP（7005.643ms）比 Pthreads（8314.071ms）快约 15.7%。该差异可能来自编译优化选项、编译参数差异（如向量化/优化级别）、运行时库开销或 CPU 亲和设置等因素，不能简单归因于编程模型本身。在多线程场景下（如 16 线程），两者性能趋于接近（548.514ms vs 617.557ms），差距缩小至约 11.2%。

加速比与并行效率：Pthreads 的加速比（16 线程：13.46x）在数值上高于 OpenMP（12.77x），但加速比受单线程基线影响较大，因此更应结合绝对运行时间与并行效率进行判断。从并行效率来看——OpenMP：$E = T_1/(P \times T_P) = 7005.643/(16 \times 548.514) = 79.8\%$，Pthreads：$E = T_1/(P \times T_P) = 8314.071/(16 \times 617.557) = 84.1\%$——两者均在合理范围内，Pthreads 效率略高，这与其手动静态划分在负载均衡场景下调度开销较小一致。

编程复杂度：虽然两者性能接近，但 OpenMP 在开发效率上具有显著优势。Pthreads 需要约 100 行代码手动管理线程创建、参数传递、任务划分（处理余数行）和线程汇合；而 OpenMP 仅需一行 `#pragma omp parallel for` 编译制导语句即可实现等价功能。此外，OpenMP 的调度策略可通过 `schedule` 子句灵活切换（静态、动态等），而 Pthreads 需要手动修改任务分配逻辑。

小结：对于矩阵乘法这类计算密集型、负载均衡的任务，本实验数据中 OpenMP 与 Pthreads 在性能上差异不大（约 10%–16%），但 OpenMP 在代码简洁性和可维护性上具有明显优势，是此类场景的首选方案。